

Лабораторная работа № 2

Задание: Перемножить 2 квадратные матрицы размера 2048x2048 с элементами типа `double complex` (комплексное число двойной точности).

Исходные матрицы генерируются в программе (случайным образом либо по определенной формуле) либо считываются из заранее подготовленного файла.

Оценить сложность алгоритма по формуле $c = 2n^3$, где n - размерность матрицы.

Оценить производительность в MFlops, $p = c/t \cdot 10^{-6}$, где t - время в секундах работы алгоритма.

Выполнить 3 варианта перемножения и их анализ и сравнение:

1-й вариант перемножения - по формуле из линейной алгебры.

2-й вариант перемножения - результат работы функции `cblas_zgemm` из библиотеки BLAS (рекомендуемая реализация из Intel MKL)

3-й вариант перемножения - оптимизированный алгоритм по вашему выбору, написанный вами, производительность должна быть не ниже 30% от 2-го варианта

Листинг:

```
#include <stdio.h>
#include <complex>
#include <windows.h>
#include <cblas.h>
#include <random>
#include <cmath>
#include <omp.h>
#include <openblas_config.h>

using namespace std;

using Complex = complex<double>;

int main() {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    int num_cores = omp_get_max_threads();
```

```

openblas_set_num_threads(num_cores);

LARGE_INTEGER freq, start, end;

const int N = 2048;

printf("Умножение матриц: %dx%d (double complex)\n", N, N);
printf("Теоретическая сложность:  $2 * N^3 = 2.0 * N * N * N$  операций\n\n",
      N, 2.0 * N * N * N);

Complex* A = new Complex[N * N];
Complex* B = new Complex[N * N];
Complex* C1 = new Complex[N * N];
Complex* C2 = new Complex[N * N];
Complex* C3 = new Complex[N * N];

printf("Генерация случайных матриц A и B (%d x %d)\n", N, N);

random_device rd;
mt19937 gen(rd());

uniform_real_distribution<double> dis(-1.0, 1.0);

for (int i = 0; i < N * N; ++i) {

    A[i] = Complex(dis(gen), dis(gen));
    B[i] = Complex(dis(gen), dis(gen));
    C1[i] = Complex(0.0, 0.0);
    C2[i] = Complex(0.0, 0.0);
    C3[i] = Complex(0.0, 0.0);
}

QueryPerformanceFrequency(&freq);

double ops = 2.0 * N * N * N;

printf("1-й вариант: Классический алгоритм\n");

QueryPerformanceCounter(&start);

for (int i = 0; i < N; ++i) {
    for (int k = 0; k < N; ++k) {
        Complex aik = A[i * N + k];
        for (int j = 0; j < N; ++j) {

```

```

        C1[i * N + j] += aik * B[k * N + j];
    }
}
}

```

```

QueryPerformanceCounter(&end);

```

```

double time1 = static_cast<double>(end.QuadPart - start.QuadPart) /
freq.QuadPart;

```

```

double mflops1 = ops / time1 / 1e6;

```

```

printf("Время выполнения: %.3f сек\n", time1);
printf("Производительность: %.2f MFLOPS\n\n", mflops1);

```

```

printf("2-й вариант: BLAS\n");

```

```

double alpha[] = { 1.0, 0.0 };
double beta[] = { 0.0, 0.0 };

```

```

QueryPerformanceCounter(&start);

```

```

cblas_zgemm(
    CblasRowMajor,
    CblasNoTrans,
    CblasNoTrans,
    N,
    N,
    N,
    alpha,
    static_cast<void*>(A),
    N,
    static_cast<void*>(B),
    N,
    beta,
    static_cast<void*>(C2),
    N
);

```

```

QueryPerformanceCounter(&end);

```

```

double time2 = static_cast<double>(end.QuadPart - start.QuadPart) /
freq.QuadPart;

```

```

double mflops2 = ops / time2 / 1e6;

```

```

printf("Время выполнения: %.3f сек\n", time2);
printf("Производительность: %.2f MFLOPS\n\n", mflops2);

printf("3-й вариант: Блочный алгоритм\n");

QueryPerformanceCounter(&start);

const int BS = 64;

#pragma omp parallel for schedule(static)
for (int ii = 0; ii < N; ii += BS) {

    for (int kk = 0; kk < N; kk += BS) {

        for (int jj = 0; jj < N; jj += BS) {

            for (int i = ii; i < ii + BS && i < N; ++i) {

                for (int k = kk; k < kk + BS && k < N; ++k) {

                    Complex aik = A[i * N + k];

                    for (int j = jj; j < jj + BS && j < N; ++j) {

                        C3[i * N + j] += aik * B[k * N + j];

                    }

                }

            }

        }

    }

}

QueryPerformanceCounter(&end);

double time3 =
    static_cast<double>(end.QuadPart - start.QuadPart) / freq.QuadPart;

double mflops3 = ops / time3 / 1e6;

double percent_of_blas = (mflops3 / mflops2) * 100.0;

printf("Время выполнения: %.3f сек\n", time3);
printf("Производительность: %.2f MFLOPS\n", mflops3);

```

```
printf("Производительность относительно BLAS: %.1f%%\n\n",  
percent_of_blas);
```

```
printf("Проверка точности:\n");
```

```
Complex* Delta = new Complex[N * N];
```

```
double minus_one[] = { -1.0, 0.0 };
```

```
cblas_zcopy(  
    N * N,  
    static_cast<void*>(C2),  
    1,  
    static_cast<void*>(Delta),  
    1  
);
```

```
cblas_zaxpy(  
    N * N,  
    minus_one,  
    static_cast<void*>(C1),  
    1,  
    static_cast<void*>(Delta),  
    1  
);
```

```
double norm = cblas_dznrm2(  
    N * N,  
    static_cast<void*>(Delta),  
    1  
);
```

```
printf("Норма разности (Классический - BLAS): %.2e\n", norm);
```

```
cblas_zcopy(  
    N * N,  
    static_cast<void*>(C2),  
    1,  
    static_cast<void*>(Delta),  
    1  
);
```

```
cblas_zaxpy(  
    N * N,  
    minus_one,
```

```

        static_cast<void*>(C3),
        1,
        static_cast<void*>(Delta),
        1
    );

    norm = cblas_dznrm2(
        N * N,
        static_cast<void*>(Delta),
        1
    );

    printf("Норма разности (Новый способ - BLAS): %.2e\n", norm);

    if (percent_of_blas >= 30.0) {
        printf("Требование выполнено!\n\n");
    }
    else {
        printf("Скорость ниже 30%% от BLAS.\n\n");
    }

    printf("ФИО: Павленко Кирилл Дмитриевич\n");
    printf("Группа: 020303-АИСа-о25\n");

    delete[] Delta;
    delete[] A;
    delete[] B;
    delete[] C1;
    delete[] C2;
    delete[] C3;

    return 0;
}

```

Комментарии к коду:

Первый вариант реализует классический алгоритм умножения матриц по формуле линейной алгебры.

Второй вариант использует функцию `cblas_zgemm` из библиотеки BLAS. Эта реализация является высокооптимизированной и использует внутренние механизмы ускорения вычислений, такие как многопоточность, эффективная работа с кэш-памятью и низкоуровневые оптимизации

процессора. Результаты данного варианта используются как эталон производительности.

Третий вариант представляет собой собственный оптимизированный блочный алгоритм умножения матриц с использованием OpenMP. Матрицы разбиваются на блоки фиксированного размера, которые последовательно обрабатываются в кэш-памяти процессора. Такой подход уменьшает количество обращений к оперативной памяти и повышает эффективность вычислений. Дополнительно используется распараллеливание вычислений между несколькими потоками процессора.

Примеры выполнения работы:

```
Умножение матриц: 2048x2048 (double complex)
Теоретическая сложность:  $2 * 2048^3 = 1.72e+10$  операций

Генерация случайных матриц A и B (2048 x 2048)
1-й вариант: Классический алгоритм
Время выполнения: 36.554 сек
Производительность: 469.98 MFLOPS

2-й вариант: BLAS
Время выполнения: 3.285 сек
Производительность: 5230.02 MFLOPS

3-й вариант: Блочный алгоритм
Время выполнения: 9.684 сек
Производительность: 1774.12 MFLOPS
Производительность относительно BLAS: 33.9%

Проверка точности:
Норма разности (Классический - BLAS): 9.95e-11
Норма разности (Новый способ - BLAS): 9.95e-11
Требование выполнено!

ФИО: Павленко Кирилл Дмитриевич
Группа: 020303-АИСa-o25
```

Вывод: В результате выполнения программы были получены значения времени работы и производительности для каждого алгоритма. Классический алгоритм показал наименьшую производительность, библиотека BLAS — наибольшую, а разработанный блочный алгоритм продемонстрировал значительно более высокую скорость по сравнению с классическим способом и приблизился к производительности BLAS. Также была выполнена проверка точности вычислений, показавшая совпадение результатов всех трёх методов с высокой точностью.